

Git & GitHub Cheatsheet

Two-Account SSH Setup | Core commands | Daily workflow | Uni PC guide

SSH Two-Account Setup — Home PC

Keys are stored in `~/.ssh/` and persist permanently on your home PC

<code>ssh-keygen -t ed25519 -C "email" -f ~/.ssh/id_ed25519_personal</code>	Generate personal SSH key pair
<code>ssh-keygen -t ed25519 -C "email" -f ~/.ssh/id_ed25519_study</code>	Generate study SSH key pair
<code>cat ~/.ssh/id_ed25519_personal.pub</code>	Print personal public key — paste into GitHub
<code>cat ~/.ssh/id_ed25519_study.pub</code>	Print study public key — paste into GitHub
<code>ssh -T git@github-personal</code>	Test personal account connection
<code>ssh -T git@github-study</code>	Test study account connection

`~/.ssh/config` file structure

<code>Host github-personal</code>	Alias used instead of github.com for personal
<code>HostName github.com</code>	Actual server to connect to
<code>User git</code>	Always 'git' for GitHub
<code>IdentityFile ~/.ssh/id_ed25519_personal</code>	Which key to use
<code>Host github-study</code>	Alias for study account
<code>IdentityFile ~/.ssh/id_ed25519_study</code>	Which key to use

[TIP] Always use `github-personal` or `github-study` in remote URLs — never `github.com` directly

SSH Two-Account Setup — Uni PC (one-time per machine)

Uni PCs wipe your home folder on logout but some drives persist (commonly `H:\` — check with your IT dept if unsure). Store your SSH keys there so you don't have to redo this every session.

Step 1 — Create your `.ssh` folder on the persistent drive

<code>mkdir /h/.ssh</code>	Create <code>.ssh</code> folder on <code>H:\</code> drive (change letter if needed)
----------------------------	---

Step 2 — Generate keys directly onto the persistent drive

<code>ssh-keygen -t ed25519 -C "study@email.com" -f /h/.ssh/id_ed25519_study</code>	Generate study key onto <code>H:\</code>
<code>ssh-keygen -t ed25519 -C "personal@email.com" -f /h/.ssh/id_ed25519_personal</code>	Generate personal key onto <code>H:\</code> (optional)

Step 3 — Create the config file on the persistent drive

<code>nano /h/.ssh/config</code>	Create/edit config on <code>H:\</code> drive
----------------------------------	--

Paste this into the config file (`Ctrl+O` to save, `Ctrl+X` to exit):

<code>Host github-study</code>	Alias for study account
<code>HostName github.com</code>	Actual server
<code>User git</code>	Always 'git'
<code>IdentityFile /h/.ssh/id_ed25519_study</code>	Path to key on <code>H:\</code> drive
<code>Host github-personal</code>	Alias for personal (if needed)
<code>IdentityFile /h/.ssh/id_ed25519_personal</code>	Path to personal key on <code>H:\</code> drive

Step 4 — Tell SSH to use the config on `H:\` each session

Because Git Bash looks in `~/.ssh` by default (which gets wiped), run this at the start of each uni session to point it at `H:\` instead:

<code>export GIT_SSH_COMMAND='ssh -F /h/.ssh/config'</code>	Tell Git to use <code>H:\</code> SSH config for this session
<code>ssh -T git@github-study</code>	Test it worked — should greet you by username

[TIP] Add the export line to `/h/.bashrc` if Git Bash loads it — then it runs automatically every session

Step 5 — Add your public key to GitHub (one-time only)

```
cat /h/.ssh/id_ed25519_study.pub
```

Print the public key — copy the whole line and paste into GitHub

[TIP] GitHub Settings > SSH and GPG keys > New SSH key > Authentication Key — you only do this once per key

Clone | Push | Pull

Cloning a repo

<code>git clone git@github-personal:yourusername/repo.git</code>	Clone from personal account
<code>git clone git@github-study:studyusername/repo.git</code>	Clone from study account
<code>git clone git@github-personal:user/repo.git myfolder</code>	Clone into a custom folder name

Pushing changes

<code>git add .</code>	Stage ALL changed files
<code>git add filename.py</code>	Stage a specific file only
<code>git commit -m "message"</code>	Commit with a description
<code>git push</code>	Push to GitHub (current branch)
<code>git push origin main</code>	Push explicitly to the main branch

Pulling & checking

<code>git pull</code>	Fetch + merge latest changes from GitHub
<code>git fetch</code>	Download changes but don't merge yet
<code>git status</code>	Show changed / staged / untracked files
<code>git log --oneline</code>	Short one-line commit history
<code>git log --oneline -10</code>	Last 10 commits only
<code>git diff</code>	Show unstaged changes
<code>git diff --staged</code>	Show staged changes ready to commit

Branches

<code>git branch</code>	List all local branches
<code>git branch -a</code>	List local AND remote branches
<code>git branch new-feature</code>	Create a new branch
<code>git checkout new-feature</code>	Switch to a branch
<code>git checkout -b new-feature</code>	Create AND switch in one step
<code>git merge new-feature</code>	Merge branch into current branch
<code>git branch -d new-feature</code>	Delete branch (safe — must be merged first)
<code>git branch -D new-feature</code>	Force delete branch (even if unmerged)
<code>git push origin new-feature</code>	Push a branch up to GitHub
<code>git push origin --delete new-feature</code>	Delete a branch on GitHub

Identity & Config

Global identity — set once (your default account on this PC)

<code>git config --global user.name "Your Name"</code>	Set default username for all repos
<code>git config --global user.email "you@email"</code>	Set default email for all repos
<code>git config --global --list</code>	Show all global config settings

Per-repo identity override — run inside a study repo after cloning

<code>git config user.name "Study Name"</code>	Override name for this repo only
<code>git config user.email "study@email"</code>	Override email for this repo only
<code>git config --list</code>	Show config for the current repo

Remotes

<code>git remote -v</code>	Show remote URLs for this repo
<code>git remote add origin URL</code>	Add a remote called origin
<code>git remote set-url origin URL</code>	Change the remote URL
<code>git remote remove origin</code>	Remove the remote

Undoing Things

<code>git restore filename.py</code>	Discard unstaged changes to a file
<code>git restore .</code>	Discard ALL unstaged changes (careful!)
<code>git restore --staged filename.py</code>	Unstage a file but keep the changes
<code>git commit --amend -m "new msg"</code>	Edit the last commit message (before pushing only)
<code>git reset HEAD~1</code>	Undo last commit — keep changes unstaged
<code>git reset --soft HEAD~1</code>	Undo last commit — keep changes staged
<code>git reset --hard HEAD~1</code>	Undo last commit AND delete changes (DANGER)
<code>git revert abc1234</code>	New commit that reverses a past commit (safe for shared repos)
<code>git stash</code>	Temporarily shelve all changes
<code>git stash pop</code>	Restore shelved changes
<code>git stash list</code>	Show all stashed changes

[TIP] Never use `git reset --hard` on commits already pushed to GitHub — use `git revert` instead

Navigation & File Commands (Git Bash)

<code>pwd</code>	Print current directory — where am I?
<code>ls</code>	List files in current folder
<code>ls -la</code>	List all files including hidden ones (like <code>.gitignore</code>)
<code>cd foldername</code>	Move into a folder
<code>cd ..</code>	Go up one folder
<code>cd ~</code>	Go to home folder
<code>cd ~/Desktop</code>	Go to Desktop
<code>mkdir foldername</code>	Create a new folder
<code>rm filename</code>	Delete a file
<code>rm -rf foldername</code>	Delete a folder and everything in it (no undo!)
<code>cat filename</code>	Print file contents to the terminal
<code>nano filename</code>	Open file in nano text editor
<code>touch filename.txt</code>	Create an empty file
<code>cp source dest</code>	Copy a file
<code>mv source dest</code>	Move or rename a file
<code>clear</code>	Clear the terminal screen

.gitignore — Keeping Files Out of GitHub

<code>touch .gitignore</code>	Create a <code>.gitignore</code> in your repo root
<code>*.pyc</code>	Ignore all compiled Python files
<code>__pycache__/</code>	Ignore Python cache folder
<code>venv/</code>	Ignore virtual environment folder
<code>.env</code>	Ignore secrets / API keys file — important!
<code>node_modules/</code>	Ignore Node.js dependencies
<code>*.log</code>	Ignore all log files
<code>git rm --cached filename</code>	Stop tracking a file already committed
<code>git rm --cached -r foldername</code>	Stop tracking a whole folder already committed

[TIP] Add `.env` to `.gitignore` BEFORE your first commit — never push API keys or passwords to GitHub

Your Daily Workflow	
<code>cd ~/Desktop</code>	1. Navigate to Desktop (or wherever)
<code>git clone</code> <code>git@github-personal:yourusername/repo.git</code>	2. Clone the repo from your personal account
<code>cd repo</code>	3. Move into the cloned folder
<code># ... edit files in VS Code ...</code>	4. Make your changes
<code>git status</code>	5. See what changed
<code>git add .</code>	6. Stage everything
<code>git commit -m "describe what you changed"</code>	7. Commit with a clear message
<code>git push</code>	8. Push back to GitHub
<code>cd .. && rm -rf repo</code>	9. Delete local copy when done

[TIP] For study repos: swap `github-personal` for `github-study`, and run `git config user.email` after cloning

Quick Reference Card			
Command	What it does	Command	What it does
<code>git init</code>	Initialise new repo	<code>git status</code>	What changed?
<code>git add .</code>	Stage all changes	<code>git add -p</code>	Stage chunk by chunk
<code>git commit -m ""</code>	Commit	<code>git push</code>	Upload to GitHub
<code>git pull</code>	Download + merge	<code>git fetch</code>	Download only
<code>git log --oneline</code>	Short history	<code>git diff</code>	Show changes
<code>git branch</code>	List branches	<code>git checkout -b x</code>	New branch + switch
<code>git merge x</code>	Merge branch	<code>git stash</code>	Shelve changes
<code>git stash pop</code>	Restore shelved	<code>git clone URL</code>	Clone a repo
<code>git remote -v</code>	Show remote URLs	<code>git tag v1.0</code>	Create a version tag
<code>git revert abc123</code>	Safe undo past commit	<code>git reset HEAD~1</code>	Undo last commit